Journal of Physics:
Conference Series

PURPOSE-LED
PUBLISHING™

PAPER • **OPEN ACCESS**

Arithmetic modeling of *k*-ary *n*-cubes and toroidal *k*-ary grids

To cite this article: Pedro J. Roig *et al* 2024 *J. Phys.: Conf. Ser.* **2701** 012036

View the article online for updates and enhancements.

You may also like

- Autoregressive Planet Search: Application to the *Kepler* Mission
Gabriel A. Caceres, Eric D. Feigelson, G. Jogesh Babu et al.
- Temporal Evolutions and Quasiperiodic Variations Present in the Sunspot Number and Group Sunspot Area Data Measured at Kodaikanal Observatory for Solar Cycles 14–24
Belur Ravindra, Partha Chowdhury, Pratap Chandra Ray et al.
- Predictive quality assurance for linear accelerator target failure using statistical process control
Jingdong Li, Dongxu Wang and Maria Chan

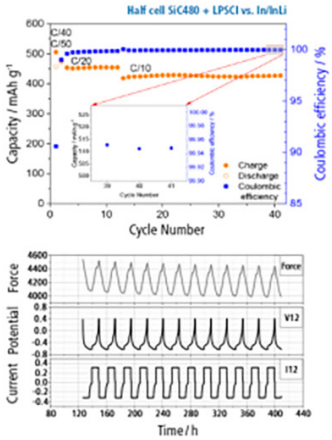
The New PAT-Cell-Solid!

EL-CELL®
electrochemical test equipment

Cycle Solid-State Batteries Under Controlled Pressure of up to 300 MPa (6 mm Diameter)!



- Adjust and measure a force of up to 9000 N on the cell stack!
Force adjustment possible throughout the entire experiment
- Built-in force, and temperature sensors!
With optional gas pressure sensor and gas in- and outlet
- PAT-Solid-Core for easy assembly and reproducible results!
Press and cycle solid-state batteries with 6 or 10 mm electrode diameter
- Cableless and highly sealed battery test cell!
For precise long-term measurements of solid-state cell chemistries



Learn more on our product website:
Scan me!

Download the data sheet (PDF):
Scan me!

Or contact us directly:
+49 40 79012-734
sales@el-cell.com
www.el-cell.com

This content was downloaded from IP address 99.233.100.81 on 24/09/2026 at 16:01

Arithmetic modeling of k-ary n-cubes and toroidal k-ary grids

Pedro J. Roig[1,2], Salvador Alcaraz[1], Katja Gilly[1], Cristina Bernad[1], Carlos Juiz[2]

[1]Miguel Hernández University, Elche (SPAIN)
[2]University of the Balearic Islands (SPAIN)

E-mail: {proig,salcaraz,katya,cbernad}@umh.es, cjuiz@uib.es

Abstract. The use of toroidal topologies offers many advantages in the computing field when it comes to both pattern spotting and path finding strategies. The former is covered by de Bruijn shapes, which permit to uniquely locate a single pattern throughout the shape. However, the latter is mainly carried out by k-ary n-cubes, which label node identifiers in a sequential order according to rows, columns, layers, and so on. This scheme facilitates the movement among nodes by just applying arithmetic operations, such as integer divisions and arithmetic modulo n. On the othe hand, toroidal k-ary grids are an alternative available in some specific cases, where determined patterns appear in all dimensions of each node, thus allowing to use those patterns to dictate paths to move among nodes. In this paper, the arithmetic bases of both path finding strategies have been presented and some pseudocode algorithms have been designed.

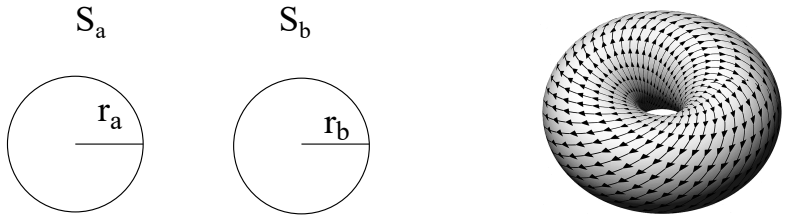
1. Introduction

Toroidal structures are relevant in different fields of mathematics, mainly in topology and geometry, but also in algebra and calculus. The most typical instance of torus is the 2D-torus, which is actually a surface, although it may be extended to further dimensions, such as a 3D-torus, whereas a circle might be considered as a 1D-torus. Focusing on topology, a torus may be seen as a closed shape with a genus on it, although more than one genus lead to the concept of double torus, triple torus and so on. In this sense, it may be said that a donut, a mag of coffee or a classic pool float are all topologically equivalent to a 2D-torus, as all of their surfaces are tori with only one genus [1]. Focusing on geometry, it is possible to represent tori in diverse manners, where each of them offers different characteristics in terms of curvature or symmetry, as well as a different perspective when it comes to visualization [2].

Sticking to a bidimensional torus, it may be seen as any topological space homeomorphic to the product of a pair of circles, thus a 2D-torus is topologically equivalent to the product of two unit circles (or unit 1-sphere $\mathbb{S}^1$), such that $\mathbb{T}^2 := \mathbb{S}^1 \times \mathbb{S}^1$. There are basically three main ways to represent a 2D-torus, where one of them uses a flat scheme in an $\mathbb{R}^4$ space, another one does a curved representation in an $\mathbb{R}^3$ space and the other one does a flat diagram in an $\mathbb{R}^2$ space [3].

The most straightforward manner is to embed each of both circles composing a 2D-torus in an independent $\mathbb{R}^2$ space, thus resulting that both circles need an $\mathbb{R}^2 \times \mathbb{R}^2 := \mathbb{R}^4$ space to be displayed. This representation is called Clifford torus [4], and it happens that if the radius of both circles is $1/\sqrt{2}$, then it perfectly fits into a unit 3-sphere ($\mathbb{S}^3$). With respect to its visualization, both circles could be separately exhibited, hence leading to a symmetric flat

Content from this work may be used under the terms of the Creative Commons Attribution 3.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.
Published under licence by IOP Publishing Ltd



(a) Two circles forming a 2D-torus (b) 3D projection

Figure 1: Clifford Torus embedded in $\mathbb{R}^4$ as two independent circles (left) and as a stereographic projection of a simple rotation (right).

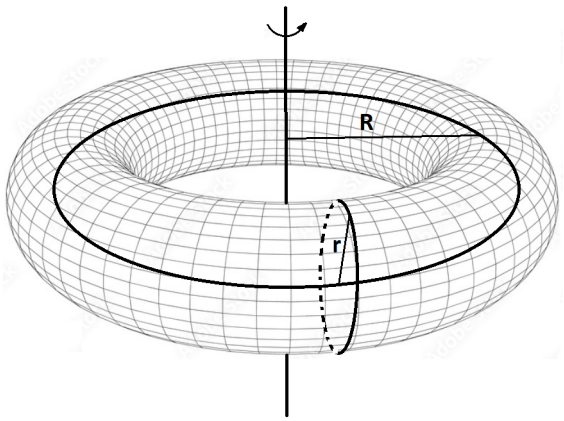


Figure 2: Ring torus embedded in $\mathbb{R}^3$ with both its radii.

diagram where circles S_a and S_b, with radii r_a and r_b, are shown independently, as viewed in Figure 1a. However, the usual way to go is to get a stereographic projection in 3D of the torus in 4D, such as that depicted in Figure 1b, where a simple rotation in $\mathbb{R}^4$ is projected into $\mathbb{R}^3$.

On the other hand, the most common representation of a 2D-torus is that where one of the circles is represented with 2 independent axes, whilst the other circle only has one. This way, the former is embedded into $\mathbb{R}^2$ and the latter is done into $\mathbb{R}^1$, thus requiring $\mathbb{R}^2 \times \mathbb{R}^1 := \mathbb{R}^3$ space to be displayed. However, it is to be noted that embedding a 2D-torus in $\mathbb{R}^3$ results in an asymmetric reduced-dimension projection related to the maximally symmetric Clifford torus, which is embedded in $\mathbb{R}^4$ [5].

The 2D-torus representation in $\mathbb{R}^3$ presents some curvature, as it has a positive Gaussian curvature in the outside, whilst it has a negative Gaussian curvature in the inside, as opposed to the flat curvature shown in its $\mathbb{R}^4$ counterpart. Hence, this approach allows to state that a 2D-torus is a surface of revolution attained when moving a circle around an axis, such that both geometrical shapes are located on the same plane [6].

This feature permits the classification of 2D-tori according to the relative position of the axis with respect to the circle, such that if it is exterior, then a ring torus is obtained, or if it is tangent, then a horn torus is attained, or if it is interior, then a spindle torus is achieved, or if it passes through the center of the circle, then a sphere is done, which is homeomorphic to $\mathbb{S}^2$ and it has no genus [7]. Centering on a ring torus, the minor radius is called section radius, which stands for that of the circle being revolved along the axis, whereas the major radius is named revolution radius, which indicates the shortest segment going from the axis to the center of the circle [8]. Figure 2 exhibits the former as r and the latter as R.

On the contrary, it is possible to transform a 2D-torus into a flat torus, thus embedding a

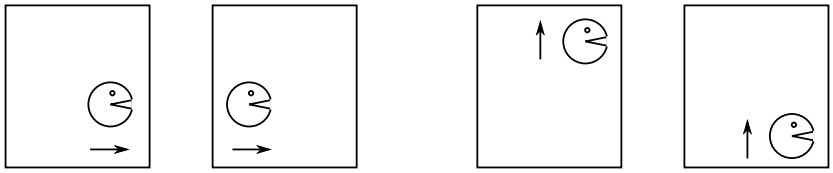


Figure 3: Movement in a flat torus embedded in $\mathbb{R}^2$, rightwards (left frames) and upwards (right frames).

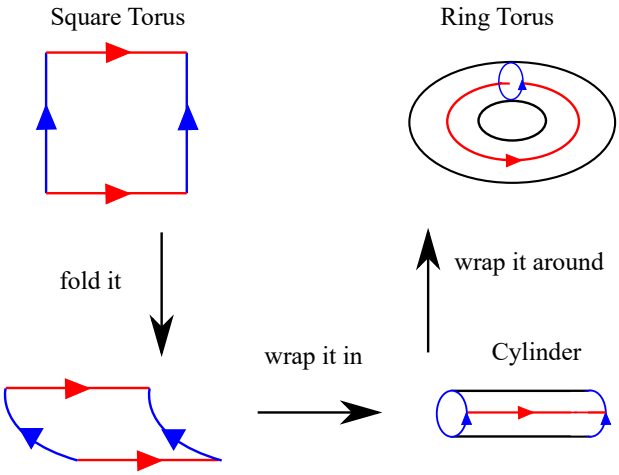


Figure 4: Process to convert a flat torus in a ring torus.

2D-torus in $\mathbb{R}^2$. A flat torus is also known as square torus, which is basically a square with oriented edges, whose concept is often employed in topology when describing the features of some surfaces like tori, Klein bottles, spheres and projective planes. Sticking to the flat torus, it may be seen as a square where its opposite edges are glued together, such that traversing one edge in a certain direction results in appearing on the opposite edge with just the same orientation [9]. A typical example of this is the PacMan universe, as exhibited in Figure 3, where getting out of the right edge gets you back on the left edge with the same orientation, as it happens in the two frames in the left, whilst getting out of the top edge gets you back at the bottom edge, as it occurs in the two frames in the right with the same orientation.

Likewise, the flat torus representation reinforces the idea of a 2D-torus being a surface ($\mathbb{T}^2 := \mathbb{S}^1 \times \mathbb{S}^1$) when dealing with its embedding in $\mathbb{R}^3$, which might sometimes get confused with the volume of a solid torus ($\mathbb{S}^1 \times \mathbb{D}^2$), meaning the torus along with the space inside. Hence, in order to further strengthen the idea of a 2D-torus being a surface, Figure 4 depicts a process to convert a flat torus in a ring torus, which also works the other way around [10]. Such a process only involves three easy steps to be applied to the flat torus to become a ring torus, such as folding, wrapping in and wrapping around.

From the three representations exposed, the one based in embedding a 2D-torus in an $\mathbb{R}^4$ space is the less intuitive, as our physical world is tridimensional, thus it is the least commonly used. Otherwise, the most intuitive one is the one based in embedding a 2D-torus in an $\mathbb{R}^3$ space, as it permits to clearly visualize the curvature of the shape, thus leading to the usual shape of a donut. However, for the purpose of this paper, the most interesting is the one based in embedding a 2D-torus in an $\mathbb{R}^2$ space, as it allows a flat representation as in the PacMac world, whose concept of identifying opposite edges so as to permit the movement between them may be used to better understand toroidal topologies of any dimension [11].

In this paper, toroidal topologies are exposed according to the approach based on flat torus, where a symmetric topology of nodes is built up throughout a toroidal shape of dimension n. With that in mind, an algorithmic model coded in pseudocode is going to be proposed for a generic k-ary n-grid and a generic toroidal k-ary grid, in a way that such algorithms minimize the Hamming distance when moving from a given source node to a particular destination node within each one of those topologies proposed.

The organization of the rest of the paper goes as follows: first, Section 2 exposes the concept of norm and distance, focusing on Hamming distance, then, Section 3 introduces the most common toroidal topologies, next, Section 4 outlines de Bruijn shapes, after that, Section 5 presents k-ary n-grids along with a model in pseudocode, afterwards, Section 6 explains toroidal k-ary grids along with a model in pseudocode, and eventually, Section 7 draws some final conclusions.

2. Norm and distance

The three types of spaces used above to embed a 2D-torus, namely $\mathbb{R}^4$, $\mathbb{R}^3$, and $\mathbb{R}^2$, are obviously all instances of $\mathbb{R}^n$. It happens that $\mathbb{R}^n$ is an Euclidean space, which implies the existence of an inner product, that being a symmetric bilinear form that is positive definite. In other words, the product of two vectors within the space is commutative and greater than zero, although yielding zero only if both vectors are the same one. It is to be noted that inner product spaces are a subset of normed vector spaces, which are a subset of metric spaces, which are a subset of topological spaces. Therefore, a norm may be defined in $\mathbb{R}^n$ in order to induce a distance, also known as its proper norm induced metric. In fact, $\mathbb{R}^n$ induces its canonical norm, given by $\|x\| = \sqrt{< x, x >}$, even though different norms could be defined, leading to different expressions for the distance function, considering that a norm is a one vector function, whilst a distance is a two vector function.

A norm always induces a distance by $d(x, y) = \|y - x\|$, whose expression will depend of the norm considered. Actually, distance in an Euclidean space may be seen as the norm of the translation vector mapping the source point to the destination point. It is possible to create a distance function with any norm, even though there are some definitions of distance which do not correspond to any norm, such as $d(x, x) = 0$ and $d(x, y) = 1$ when $x \neq y$, or $d(x, y) = |arctan(y) - arctan(x)|$, which are found in metric spaces not being normed vector spaces. As a matter of fact, a distance is a metric because it is symmetric, positive definite, and it fulfills triangle inequality, whereas a norm meets absolute homogeneity, positive definite, and triangle inequality. Additionally, as $\mathbb{R}^n$ is an inner product space, norm properties still hold, in addition to the Cauchy-Schwartz inequality, the parallelogram law and the polarization identity.

Therefore, it is to be noted that a distance function such as the one defined as $d(x, y) = \|y-x\|$ is equivalent to $d(x, y) = \|x - y\|$ due to the symmetric property, as well as $d(x, y) \geq 0$, where $d(x, y) = 0$ if and only if $x = y$ due to the positive definite property, whilst $d(x, z) \leq d(x, y) + d(y, z)$ due to the triangular inequality.

Anyway, the most usual norms to induce a distance, such as $d(x, y) = \|y - x\|$, are derived from the l_p-norm (1), where $p \in \mathbb{R}$ and x is a vector within $\mathbb{R}^n$ with n components, where x_i is a given component i, ranging from 1 to n, belonging to such a vector x [12].

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p} \quad (p \in \mathbb{R}, n \in \mathbb{N}) \tag{1}$$

To begin with, if $p = 0$, we have the l_0-norm, which results in (2).

$$\|x\|_0 = \sqrt[0]{\sum |x_i|^0} \tag{2}$$

It is to be said that l_0-norm is not a proper norm because it is not homogeneous, although

it is often referred to as a norm. Furthermore, the zeroth-root is not defined, as $\sqrt[0]{x} = x^{1/0}$ and $1/0$ is undefined, although its limit tends to either ∞ or $-\infty$, depending on the value of base x.

Hence, there is an alternative definition of the l_0-norm as the total amount of non-zero components of vector x, also known as the number of non-zero elements of vector x (3).

$$\|x\|_0 = \#\left(i \mid x_i \neq 0\right) \tag{3}$$

A typical use of the l_0-norm is compressive sensing, whose target is to reach the sparsest solution of an underdetermined linear system, meaning the option with the least non-zero entries, thus getting to minimize the l_0-norm.

Regarding the distance induced by l_0-norm, it is called Hamming distance and it represents the number of components with different values between a pair of vectors. In this sense, if two vectors x and y are involved, then the corresponding components between them, such as x_i and y_i, are compared by means of the *XOR* function, yielding a 0 for a corresponding component i of the result vector z if the values of x and y for that component are equal, thus getting $z_i = 0$, or otherwise, yielding a 1 if they are different, thus getting $z_i = 1$. Afterwards, the number of 1s of the result vector are added up, such as $\sum z_i$, which stands for the Hamming distance between x and y, such as $l_0 = \sum z_i$. The overall process to get the Hamming distance is expressed in (4).

$$\|x - y\|_0 = \sum_{i=1}^{n} \left(x_i \; XOR \; y_i\right) \tag{4}$$

Hamming distance may also be applied to a pair of strings or numbers in any base, such as strings x and y containing the same number of characters. In this context, the distance exposes the number of discordant characters between both strings, where the value of each corresponding character related to a given position in both strings are compared in due course with the same method exposed above for comparing the components of two vectors.

On the other hand, if $p = 1$, we have the l_1-norm, which results in (5):

$$\|x\|_1 = \sqrt[1]{\sum |x_i|^1} = \sum |x_i| \tag{5}$$

The l_1-norm is fairly popular and it is also known as Manhattan norm. Its distance induced is exposed in (6), which is also known as Sum of Absolute Differences (SAD).

$$\|x - y\|_1 = \sum_{i=1}^{n} |x_i - y_i| \tag{6}$$

Such a distance is often referred to as Manhattan distance, as well as taxicab distance or rectilinear distance, because of the grid pattern of Manhattan's streets in New York City, as taxi drivers can only make orthogonal movements to drive between two locations. Likewise, Manhattan distance is employed in chess so as to account for the minimal amount of orthogonal moves that a King must make to reach a given destination within the chessboard, counted up as the number of squares traversed.

Likewise, Hamming distance is branded as the Manhattan distance between strings, symbol vectors or bit vectors, because Hamming distance may be interpreted as the number of characters, symbols or bits to be changed in order to convert one string into another one.

However, the result shown for SAD is often scaled to a vector unit, as viewed in (7), which is also branded as Mean Absolute Error (MAE).

$$\frac{1}{n}\|x - y\|_1 = \frac{1}{n}\sum_{i=1}^{n} |x_i - y_i| \tag{7}$$

Furthermore, if $p = 2$, we have the l_2-norm, which results in (8):

$$\|x\|_2 = \sqrt[2]{\sum |x_i|^2} \tag{8}$$

The l_2-norm is the most popular and it is also known as Euclidean norm. It is used in all fields of Science, Mathematics, Engineering and Technology (STEM). Its distance induced is exhibited in (9), which is also known as Euclidean distance and it is a generalization of the Pythagorean Theorem. Basically, this definition of distance is the most commonly used by far, as it measures the shortest path between two points in an n-dimensional Euclidean space, which is always isomorphic to $\mathbb{R}^n$, and it is given by the magnitude of the vector joining both points.

$$\|x - y\|_2 = \sqrt[2]{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{9}$$

Besides, that expression may also be given as its squared form, as viewed in (10),which is also known as Sum of Squared Difference (SSD).

$$\|x - y\|_2^2 = \sum_{i=1}^{n} (x_i - y_i)^2 \tag{10}$$

Also, that result shown for SSD is often scaled to a vector unit, as viewed in (11), which is also branded as Mean Squared Error (MSE). This is commonly used when computing linear regression problems or least square optimization.

$$\frac{1}{n}\|x - y\|_2^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - y_i)^2 \tag{11}$$

Additionally, if $p = \infty$, we have the l_∞-norm, which results in (12):

$$\|x\|_\infty = \sqrt[\infty]{\sum |x_i|^\infty} \tag{12}$$

The l_∞-norm is fairly popular and it is also known as Chebyshev norm. However, this definition might seem hard to implement at first sight, although if we consider x_j as the highest entry of vector x, by applying the property of infinity itself, it can be stated that $x_j^\infty >> x_i^\infty \; \forall i \neq j$, thus leading to $\sum |x_i|^\infty = |x_j|^\infty$, which in turn drives the definition of the infinity norm to $\|x\|_\infty = \sqrt[\infty]{\sum |x_i|^\infty} = \sqrt[\infty]{|x_i|^\infty} = |x_j|^\infty$.

Therefore, the expression for the l_∞-norm gets simplified as it just involves the component with the higher magnitude of vector x (13):

$$\|x\|_\infty = max\left(|x_i|\right) \tag{13}$$

Its distance induced is exhibited in (14), which is also known as Chebyshev distance, Tchebychev distance or maximum metric, and it accounts for the greatest difference along any coordinate dimension.

$$\|x - y\|_\infty = \max_i \left(|x_i - y_i|\right) \tag{14}$$

Chebyshev distance is often employed to account for the most expensive element in a set of items, or in warehouse logistics when measuring the time taken by an overhead crane to displace an object, as movement along horizontal and vertical axes may be done at the same time and speed. Likewise, Chebyshev distance is used in chess so as to account for the minimal amount

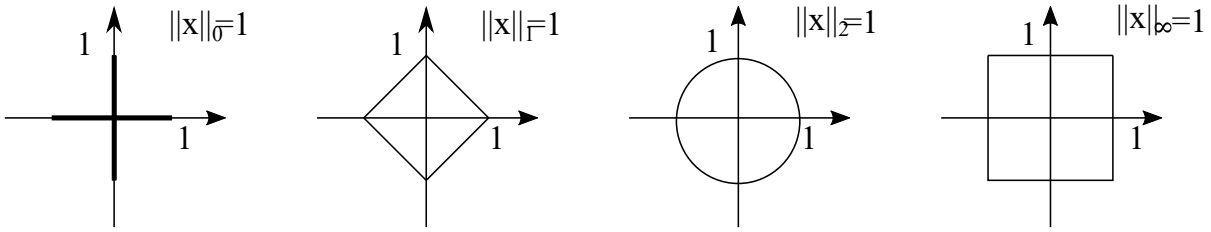


Figure 5: Displaying a unit circle in $\|x\|_0$, $\|x\|_1$, $\|x\|_2$ and $\|x\|_\infty$.

of moves, either rectilinear or diagonal, that a King must make to reach a given destination within the chessboard, counted up in number of squares traversed. In this context, it is to be noted that Manhattan distance needs two moves to reach any of the diagonal neighbors of a given square, whereas Chebyshev distance needs just one, whilst on the contrary, both metrics need only one move to get to any of the orthogonal neighbors.

In summary, given two vectors $v = \{v_1, v_2\}$ and $w = \{w_1, w_2\}$, then these distances apply:

- $\|v - w\|_0 = (v_1 \; XOR \; w_1) + (v_2 \; XOR \; w_2)$
- $\|v - w\|_1 = |v_1 - w_1| + |v_2 - w_2|$
- $\|v - w\|_2 = \sqrt[2]{(v_1 - w_1)^2 + (v_2 - w_2)^2}$
- $\|v - w\|_\infty = max\,(|v_1 - w_1|, |v_2 - w_2|)$

Moreover, when it comes to comparing numbers expressed in a given base, the value of the Hamming distance between them depends on the base those number are expressed in. For instance, setting node $a = 3$ and node $b = 4$, then the Hamming distance between a and b in decimal (Base-10 system) accounts for $\|a_{B\text{-}10} - b_{B\text{-}10}\|_0 = \|3 - 4\|_0 = (3 \; XOR \; 4) = 1$, whilst in binary (Base-2 system) accounts for $\|a_{B\text{-}2} - b_{B\text{-}2}\|_0 = \|011 - 100\|_0 = (0 \; XOR \; 1) + (1 \; XOR \; 0) + (1 \; XOR \; 0) = 1 + 1 + 1 = 3$, whereas in ternary (Base-3 system) accounts for $\|a_{B\text{-}3} - b_{B\text{-}3}\|_0 = \|10 - 11\|_0 = (1 \; XOR \; 1) + (1 \; XOR \; 0) = 0 + 1 = 1$.

Eventually, in order to visually compare the effect of each of the distances studied, Figure 5 displays a unit circle, defined as $\mathbb{S}^1 = \{(x_1, x_2) \in \mathbb{R}^2 : x_1^2 + x_2^2 = 1\}$ when applying the l_0-norm (the one where Hamming distance is induced from), the l_1-norm (the one where Manhattan distance is induced from), the l_2-norm (the one where Euclidean distance is induced from) and the l_∞-norm (the one where Chebyshev distance is induced from), respectively.

3. Classification of toroidal topologies

In order to perform a classification of toroidal topologies, the embedding representation of a flat torus is going to be used due to its flatness when dealing with 2D-tori and the easiness to get it extended to higher dimensions. In this sense, a toroidal topology could be denoted as an n-dimensional matrix, where each of those dimensions is composed of k nodes. Every node in that matrix have two links per dimension, where one is linked to its predecessor node and the other one is done to its successor node in each dimension. In this sense, nodes located on an edge of any dimension have a wraparound link to connect to its corresponding opposite edge in the same dimension. Hence, each node has an overall amount of $2n$ nodes, unless there is just one node in a given dimension, where obviously there would not be any link associated to such a dimension, although there would still be 2 links for each dimension with more than one node.

Anyway, Figure 6 depicts a some instances of toroidal topologies in one, two and three dimensions, where wraparound links close each structure in every dimension. First of all, there is a linear toroidal architecture, where nodes are connected along an array of lenght 4, with 4 nodes connected in a daisy chain manner. Besides, there is a bidimensional toroidal design,

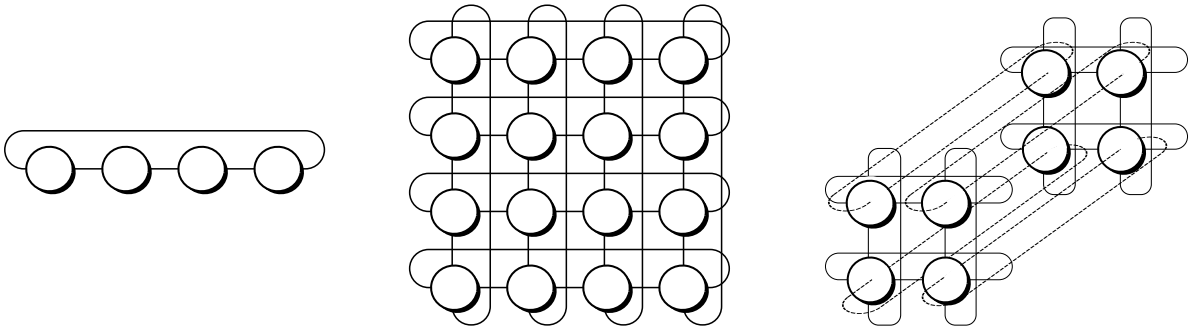


Figure 6: Toroidal topologies in 1D, 2D and 3D, respectively.

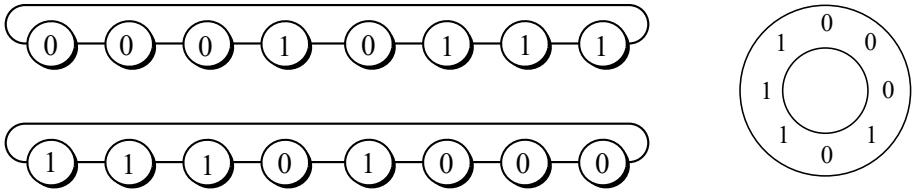


Figure 7: Available instances of de Bruijn sequence $B(2,3)$.

where nodes are distributed across a 4×4 matrix, with 4 nodes contained in each row and each column. Moreover, there is a tridimensional toroidal scheme, where nodes are located in the form of a 3D-hypermatrix of dimensions $2 \times 2 \times 2$, with 2 nodes included in each row, each column and each layer [13].

Having exposed the physical structure of a toroidal topology, different logical layouts may be considered in order to identify the nodes within the design. Basically any logical arrangement could be setup, although some of them have a specific interest, such as facilitating pattern spotting or optimizing path finding. Regarding the former, de Bruijn shapes permit to uniquely identify a given pattern within the shape. With respect to the latter, k-ary n-grids and toroidal k-ary grids offer convenient node identification schemes in order to implement efficient algorithms so as to get optimized paths to move among nodes and facilitate resource migration.

4. Outline of de Bruijn shapes

To begin with, de Bruijn sequences are strings composed of k^n characters or numbers where each available substring of length n, being built up with a k-ary alphabet, appears exactly once [14]. Such strings were first designed for binary alphabets, where $k = 2$, thus offering only the binary values 0 or 1, although this concept was soon extended to any k-ary alphabet. It is to be considered that rotating an instance of de Bruijn sequence do not create a new instance, even though reflecting an instance does create a new one.

The notation of a de Bruijn sequence is given by $B(k, n)$, where its design may be exposed as a toroidal array, with a wraparound link between the first and the last element, or otherwise, in a circular manner. Actually, Figure 7 depicts the two available instances of the de Bruijn sequence $B(2, 3)$, where one instance is 00010111 and the other one is its own reflection, which obviously is 11101000. Besides, the toroidal array representation is shown on the left of the picture, whilst the circular fashion is viewed on the right.

Taking one of the instances, such as the string 11101000, it is possible to check that all available substrings of length $n = 3$ and alphabet $k = 2$. In fact, if the leftmost element of each substring is considered as its handle. In turn, if that handle is moved from the leftmost position of the string, which may be assigned to 1, towards the rightmost position of the string, which

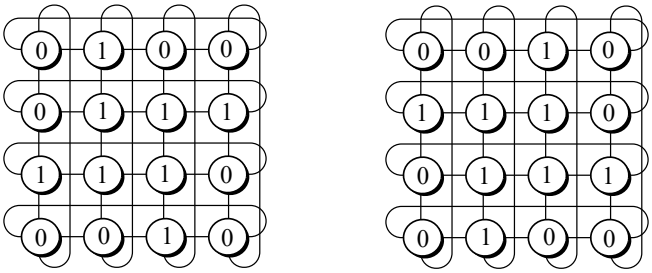


Figure 8: Available instances of de Bruijn torus $(4, 4; 2, 2)_2$.

may be assigned to 8, as $2^3 = 8$, then every possible combination of substrings with $n = 3$ and $k = 2$ are attained precisely once. All those available substrings are exhibited in Table 1.

Table 1: Substrings within a de Bruijn sequence $B(2, 3)$: 11101000.

Handle 1	**Handle 2**	**Handle 3**	**Handle 4**	**Handle 5**	**Handle 6**	**Handle 7**	**Handle 8**
111	110	101	010	100	000	001	011

Therefore, spotting a certain substring gives information about the linear location of a pattern within a toroidal array, or otherwise, the angular location of a pattern within a circular array.

On the other hand, de Bruijn sequences may be extended to higher dimensions in order to expand the capabilities of pattern spotting [15]. In this sense, if two dimensions are considered, then a de Bruijn torus is defined as a toroidal bidimensional matrix of dimensions $r_1 \times r_2$, where all available bidimensional submatrices of dimensions $m_1 \times m_2$, being constructed with a k-ary alphabet, appears exactly once. The notation of de Bruijn tori are $(r_1, r_2; m_1, m_2)_k$. Both the matrix and its submatrices may be either square or rectangular, depending if the values of both r or both m are equal or otherwise.

Figure 8 depicts the two available instances of the de Bruijn torus $(4, 4; 2, 2)_2$, where the nodes filled with ones reveal the shape of the Bridget's cross. Actually, the picture on the left shows the clockwise instance, whereas the picture on the right does the counterclockwise instance, which happens to be the transpose matrix of the other one.

Regarding the handle to identify each submatrix within the matrix, the top left element is selected as its handle. Actually, Figure 9 displays the $(4, 4; 2, 2)_2$ clockwise matrix on the left, along with the $(4, 4; 2, 2)_2$ counterclockwise matrix on the right, where a mapping of each available submatrix for each case have been carried out. In fact, the picture exhibits every possible 2×2 submatrix built up with a binary alphabet within the 4×4 matrix, where each of those submatrices appear precisely once in both instances, even though they are obviously located in different locations because they are indeed different toroidal matrices.

Additionally, de Bruijn tori may be further extended into higher dimensions, leading to de Bruijn hypertori of n dimensions. In those cases, the notation is extended as $(r_i; m_i)_k$, where i denote the different dimensions involved in both the hypermatrix and the hypersubmatrices. Setting to focus in 3D, the shapes of a de Bruijn 3D-hypermatrix, as well as its corresponding 3D-hypersubmatrices may be either cubic, square prism or rectangular prism, depending on the number of equal values of r_i or m_i. Anyway, the smallest instance is $(16, 4, 4; 2, 2, 2)_2$, which accounts for 256 different cubic patterns appearing just once within the toroidal square prism.

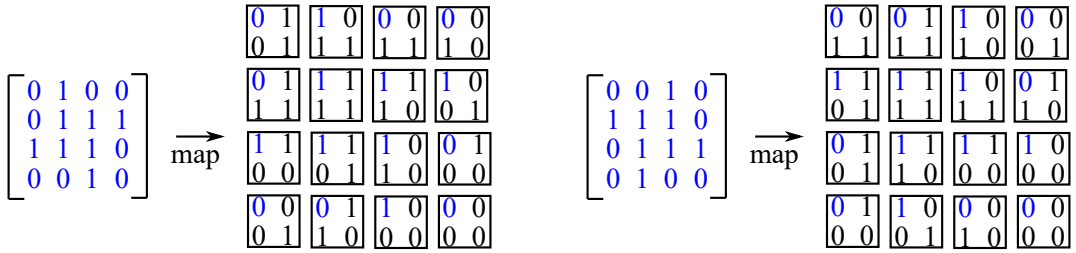


Figure 9: Submatrices within both instances of a de Bruijn torus $(4, 4; 2, 2)_2$.

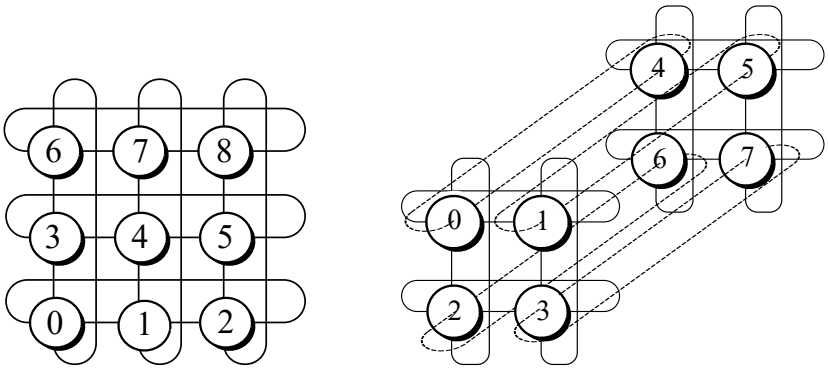


Figure 10: k-ary n-cube with $k = 3$ and $n = 2$ (left) and $k = 2$ and $n = 3$ (right), both with their nodes in decimal notation.

5. k-ary n-cubes

A k-ary n-cube is a toroidal structure composed of n dimensions, where each of those contains k nodes. If $k = 2$, then the structure is an n-hypercube, where nodes are located right on each corner. However, if $k > 2$, then nodes are equally distributed along the whole shape [16].

When facing the implementation of a node identification scheme in a toroidal topology with the shape of a k-ary n-cube, the most straightforward method is to do it sequentially. The simplest way to do so is to initially identify each node with a natural number starting from a given corner, which will be assigned number zero. In turn, identify all nodes in a single column, where each node in located in a different row within that column, and assign them a natural number consecutively. Then, identify all nodes in each column in a single layer, and assign them natural numbers in an ordered fashion. After that, identify all nodes in all rows in each layer in a tridimensional structure in order to assign them natural numbers accordingly, and so on.

Applying this scheme allows to take advantage of integer divisions by k and arithmetic modulo k for bidimensional deployments, along with its powers for higher dimensions, in order to achieve the best way to go from a given source node to a given destination node by just using those arithmetic operations, which in turn, may lead to the design of an efficient algorithm with arithmetic operations so as to move between any pair of nodes. Figure 10 depicts a 3-ary 2-cube on the left and a 2-ary 3-cube on the right with nodes in decimal notation.

An equivalent manner to assign node identifiers in a k-ary n-cube is to use the same strategy by expressing the natural numbers in a base-k system, in a way that each identifier has n symbols, where each of those is associated to one of the dimensions involved. In other words, the point is to allocate a string composed of as many digits as the number of dimensions n to each node, where each of those digits may range from 0 up to $k - 1$.

In that case, one of the corners is selected as the origin, hence all its d_i digits will be set to zero. From that point on, taking a random node d, whose value related to a given dimension i is

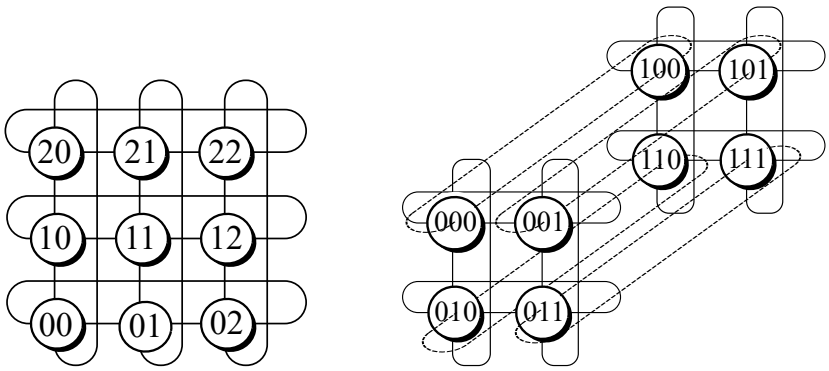


Figure 11: k-ary n-cube with $k = 3$ and $n = 2$ (left) and $k = 2$ and $n = 3$ (right), both with their nodes in its k-ary notation.

d_i, then the node attached to the left in that dimension i is assigned to the predecessor modulo k of the value d_i, whilst the node attached to the right is assigned to the successor modulo k of the value d_i, as expressed in (15). However, if the binary alphabet is employed, then such an expression gets simplified because there are only two neighbors in each dimension, as in (16).

$$(d_i \pm 1)\ modulo\ k = (d_i \pm 1)_{|k} \tag{15}$$

$$(b_i + 1)\ modulo\ k = (b_i + 1)_{|k} \tag{16}$$

The application of this alternative scheme also allows to use integer divisions and arithmetic modulo k in order to establish a moving strategy among nodes, although it also permits to implement a logic approach by comparing the corresponding symbols between source and destination in order to manage the movement between a source node and a destination node by employing *XOR* logic operations, thus leading to design an efficient algorithm based on them.

Figure 11 exhibits a 3-ary 2-cube on the left hand side and a 2-ary 3-cube on the right hand side. In both cases, the node notation is given in base k, although it could easily be converted into decimal notation. It is to be noted that node identification symbols associated with each dimension begin with zero at the origin and grow onwards regarding its relative position per dimension with respect to the origin.

With regards to the former, it involves two dimensions, as $n = 2$, thus having the shape of a square, with three nodes per dimension, as $k = 3$. In this case, the origin has been set on the bottom left corner, with the first dimension tied to the horizontal direction, representing the rows, and the second one tied to the vertical direction, representing the columns. Hence, each node is identified with $n = 2$ symbols, with values going from 0 up to $k-1$, where the rightmost symbol is associated with its position on the first dimension and the leftmost symbol is done with its position on the second dimension.

With respect to the latter, it involves three dimensions, as $n = 3$, thus having the shape of a cube, with two nodes per dimension, as $k = 2$. In that case, the origin has been set on the front top left corner, where the first and second dimensions are associated to horizontal and the vertical, respectively, whilst the third one is done to the number of layer. Therefore, every node is identified with $n = 3$ bits, as their values range from 0 to $k-1 = 1$, where the rightmost bit is tied to its position on the first dimension, the middle bit is done to its position on second one, and the leftmost bit is done to its position on the third one.

Focusing on the arithmetic approach, which is the most convenient when dealing with node identifiers in decimal notation, it is possible to extract the different symbols corresponding to

the base-k notation by applying expression (17) on decimal number a, where $\lfloor a/z \rfloor$ stands for the quotient of the integer division of a by z, whereas $a_{|z}$ does for the remainder of the integer division of a by z, meaning a modulo z, and j takes all values from 0 to $k-1$. It is to be noted that j, ranging from 0 to $n-1$, is an auxiliar variable to cover dimensions i, going from 1 to n.

$$a = \sum_{j=0}^{n-1} \left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \times k^j = \sum_{j=0}^{n-1} a_j \times k^j \tag{17}$$

On the other hand, if two node identifiers are to be compared, such as a and b, then it is possible to extract each of their corresponding pairs of symbols in base k aligned with dimension i, such as a_i and b_i, and in turn, compare them symbol by symbol accordingly. Hence, for the matching pairs of symbols, there is no movement to be made as both symbols share the same value, whereas for the mismatching pairs of symbols, then the gap between a_i and b_i needs to be shortened, where gap is defined herein as the minimum number of sequential digits modulo k to go from a_i and b_i, and to do so, that gap is confronted with half the value of n, namely $n/2$.

This way, if the gap between a given pair of symbols a_i and b_i is lower to $n/2$, then the source node must move in the direction towards the destination node with respect to the dimension i, which is the one associated with the symbol involved. On the other hand, if the gap is higher to $n/2$, then the source node must move in the direction opposite to the destination node with respect to the dimension i in order to take advantage of the toroidal nature of the whole structure. On the contrary, if the gap is equal to $n/2$, then both paths have equal cost, so either path may be taken, or otherwise, both paths may be used in order to get redundancy. This particular case where the gap is just $n/2$ could be implemented by a function called SelectPath(p,q), which chooses any of the redundant paths available, p or q. Also, if the gap is zero, no move is made.

Additionally, it is to be noted that, as we are dealing with toroidal shapes, n is the overall gap to move around a given dimension, thus leading to reach the original node after a 360 degree turn. Hence, if that gap is larger than $n/2$, then it is effectively as being smaller than $-n/2$ in absolute value. Likewise, this reasoning is analogous for gaps larger than $-n/2$ in absolute value, which are effectively smaller than $n/2$.

It is possible to craft an algorithm to move from a source node a to a destination node b by using either an arithmetic approach or a logic approach. Regarding the arithmetic one, nodes are expressed in decimal notation and every pair of symbols occupying the same position in both node identifiers are compared in order to check if the are unequal or equal. It is to be noted that symbol extraction in each node is carried out by applying the expression quoted above. With respect to the logic one, nodes are expressed in k-ary notation and the comparisons among the corresponding pairs of symbols in both node identifiers are carried out by applying the logic XOR operation, which yields 1 if a pair of symbols mismatch, or 0 if they match. Obviously, both approaches are equivalent, as both yield the same result, although the only difference between them is the step where the symbol confrontation is done. In this sense, the arithmetic approach to compare each corresponding pair of symbols is cited in (18), whilst the logic approach is quoted in (19), hence, (20) reiterates the equivalence between both expressions.

$$\begin{aligned} &\text{for } j = 0 \text{ to } k-1: \\ &\quad \text{if } \left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \neq \left\lfloor \frac{b}{k^j} \right\rfloor_{|k}: \\ &\quad\quad \text{move } a_j \text{ towards } b_j \end{aligned} \tag{18}$$

$$\begin{gathered}\text{for } j = 0 \text{ to } k-1: \\ \text{if } \left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \; XOR \; \left\lfloor \frac{b}{k^j} \right\rfloor_{|k} : \\ \text{move } a_j \text{ towards } b_j\end{gathered} \tag{19}$$

$$\left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \neq \left\lfloor \frac{b}{k^j} \right\rfloor_{|k} \iff \left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \; XOR \; \left\lfloor \frac{b}{k^j} \right\rfloor_{|k} = 1 \tag{20}$$

In order to model k-ary n-cube topologies by means of an algorithm, it may be designed to apply it to a given node in a distributed manner or in a centralized fashion. The former makes decisions based on its own information, which results in just moving towards its neighbor located in a dimension where there is a mismatching pair of symbols between that node and the destination node. Therefore, only one move is made by the distributed algorithm. In this case, the proper thing to do is to sweep all dimensions i in order to search for the cases where $a_i \neq b_i$, which may be done either by the arithmetic method or the logic method. However, in order to avoid checking each dimension in a sequential order, hence favoring lower dimensions as they always checked earlier than their higher counterparts, then a random array for the indexes of dimensions might be implemented so as to get a fairer ordering of the dimensions to be checked.

On the contrary, the centralized algorithm makes the whole movement from the source node to the destination node, and not just one move as the distributed algorithm does. Also, in this case it is not necessary to randomize the order of checking dimensions as this algorithms implies as many moves as necessary to reach destination, so dimensions could be checked sequentially.

Hence, algorithm 1 models a distributed algorithm, whose execution only depends on the running node, whereas algorithm 2 models a centralized algorithm, whose execution is made by a centralized controller. It is to be noted that the extraction and comparison of the corresponding pairs of symbols between node a and node b is shown with the arithmetic approach, even though it could be easily substituted with the logic approach, as exposed above. Besides, the distributed algorithm takes a as the current node, because the real source node does not offer relevant information, as it happens with the routers when dealing with incoming packets to be routed towards their destination. However, the centralized algorithm does take a as the source node. Additionally, both algorithms take b as the destination node.

6. Toroidal k-ary grids

An alternative layout to assign node identifiers in toroidal topologies are toroidal k-ary grids, although it is only available in a handful of specific cases. This type of scheme is not as straightforward to design as k-ary n-cubes, although it offers an advantage from an operational point of view when $k \geq 4$. Sticking to those cases in k-ary n-cubes, if one given node is chosen, then there is not a direct connection to all nodes whose Hamming distances is one, which it does happen when $k = 2$ or $k = 3$, as seen in previous pictures.

For instance, Figure 12 depicts a 4-ary 2-cube in decimal notation on the left and in 4-ary notation on the right. Then, the left picture allows to see that not all nodes along a single line are directly connected, where horizontal lines account for the first dimension and vertical lines do for the second dimension. This fact may be seen in the right picture, which also offers another perspective, as not all nodes sharing the leftmost symbol or the rightmost symbol have a direct connection. Hence, some nodes with only one discordant symbol are not one hop away.

Therefore, toroidal k-ary grids come into play in order to allocate all nodes with just one mismatching symbol at only one hop away, thus equating the Hamming distance between any couple of nodes with the number of hops away to reach one another. This way, for any dimension

Algorithm 1 Distributed algorithm to move from node a to node b within a k-ary n-cube.

Require: a, b, k, n
  $flag \leftarrow 0$
  for $j \leftarrow 0$ to $n-1$ **do**
    while $flag == 0$ **do**
      $a_j \leftarrow \left\lfloor \frac{a}{k^j} \right\rfloor_{|k}$
      $b_j \leftarrow \left\lfloor \frac{b}{k^j} \right\rfloor_{|k}$
      $gap \leftarrow b_j - a_j$
      if $gap\ != 0$ **then**
        if $gap > {}^{n}/_{2}$ **then**
          $gap \leftarrow -n + gap$
        else if $gap \leq -{}^{n}/_{2}$ **then**
          $gap \leftarrow n + gap$
        end if
        if $gap < 0$ **then**
          $a_j \leftarrow (a_j - 1)_{|k}$
        else if $gap > 0$ **then**
          $a_j \leftarrow (a_j + 1)_{|k}$
        else if $gap == {}^{n}/_{2}$ **then**
          SelectPath $(a_j \leftarrow (a_j - 1)_{|k}\ ,\ a_j \leftarrow (a_j + 1)_{|k})$
        end if
        $flag \leftarrow 1$
      end if
    end while
  end for

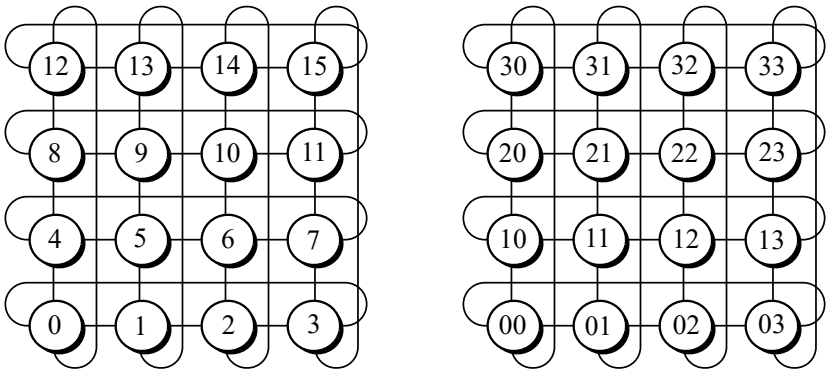


Figure 12: k-ary n-cube with $k = 4$ and $n = 2$ in decimal notation (left) in its k-ary notation (right).

i, taking nodes a and b, then its Hamming distance is given by (21), where the mismatching pairs of symbols indicate the dimensions where a move must take place to go from a to b.

$$Hamming\ distance = \sum_{j=0}^{n-1} \left(\left\lfloor \frac{a}{k^j} \right\rfloor_{|k} \neq \left\lfloor \frac{b}{k^j} \right\rfloor_{|k} \right) = \sum_{j=0}^{n-1} \left(\left\lfloor \frac{a}{k^j} \right\rfloor_{|k}\ XOR\ \left\lfloor \frac{b}{k^j} \right\rfloor_{|k} \right) \tag{21}$$

The implementation of toroidal k-ary grids must distinguish the parity of n, as even values result in bidimensional implementations, whereas odd values result in k-dimensional schemes.

Algorithm 2 Centralized algorithm to move from node a to node b within a k-ary n-cube.

Require: a, b, k, n
  for $j \leftarrow 0$ to $n-1$ **do**
    $a_j \leftarrow \left\lfloor \frac{a}{k^j} \right\rfloor_{|k}$
    $b_j \leftarrow \left\lfloor \frac{b}{k^j} \right\rfloor_{|k}$
    $gap \leftarrow b_j - a_j$
    if $gap \,!= 0$ **then**
      if $gap > {}^{n}/_{2}$ **then**
        $gap \leftarrow -n + gap$
      else if $gap \leq -{}^{n}/_{2}$ **then**
        $gap \leftarrow n + gap$
      end if
      while $gap \,!= 0$ **do**
        if $gap < 0$ **then**
          $a_j \leftarrow (a_j - 1)_{|k}$
          $gap \leftarrow gap + 1$
        else if $gap > 0$ **then**
          $a_j \leftarrow (a_j + 1)_{|k}$
          $gap \leftarrow gap - 1$
        else if $gap == {}^{n}/_{2}$ **then**
          SelectPath $(a_j \leftarrow (a_j - 1)_{|k}\ ,\ a_j \leftarrow (a_j + 1)_{|k})$
        end if
      end while
    end if
  end for

In all cases, definite patterns of symbols are spotted along different dimensions, which allows to visually guess the path, or paths, to move from a source to a destination node by following the lines where the patterns of the mismatching symbols are located. It is to be said that a value x found within a pattern within a given line going along a certain dimension means that such a symbol do not follow any definite pattern in that dimension line, whereas a numeric value in in a given position within a pattern stands for a symbol being common in the same position for all node identifiers sharing that dimension line [17].

Figure 13 exhibits three toroidal grids, each one with its own features. The left picture has an even value for n, as $n = 4$, and $k = 2$, which accounts for a binary alphabet. In that case, all nodes are expanded onto a toroidal bidimensional grid, where each node is identified by $n = 4$ symbols taken from the binary alphabet, and patterns are spotted in the horizontal and vertical dimensions. The middle picture has an odd value for n, as $n = 3$, and $k = 2$, which also uses a binary alphabet. In this case, nodes are extended onto a toroidal tridimensional grid, because $n = 3$, where nodes are branded by $n = 3$ binary symbols, and patterns are located in the horizontal, vertical and depth dimensions. The right picture has an even value for n, as $n = 2$, and $k = 3$, which stands for a ternary alphabet. In such a case, the nodes are organized onto a toroidal bidimensional grid, where every node is labeled by $n = 2$ ternary symbols, and patterns are found in the horizontal and vertical dimensions.

Likewise, Figure 14 depicts a toroidal n-dimensional k-ary grid, which has an odd value for n, as $n = 3$, and $k = 3$, which means a ternary alphabet. Then, all nodes are distributed onto a toroidal tridimensional grid, where each node is named by $n = 3$ symbols taken from the ternary alphabet, and patterns are seen in the horizontal, vertical and depth dimensions.

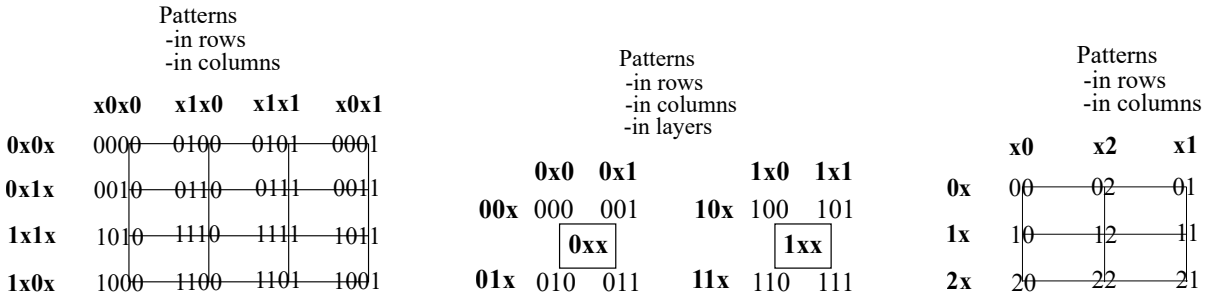


Figure 13: Toroidal bidimensional binary grid for $n = 4$ (left), toroidal n-dimensional binary grid for $n = 3$ (center) and toroidal n-dimensional k-ary grid for $k = 3$ and $n = 3$ (right).

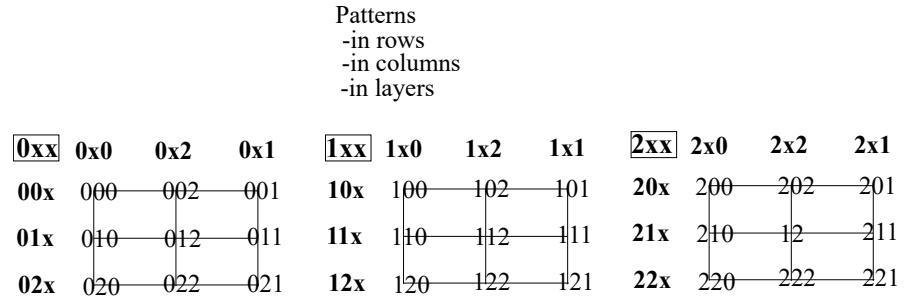


Figure 14: Toroidal n-dimensional k-ary grid for $k = 3$ and $n = 3$.

In order to model toroidal k-ary grids, the algorithms are similar to the ones exposed for the k-ary n-cubes, although the way to move among nodes does not depend on an arithmetic operation. On the contrary, the location of each node is associated to as many patterns as the number of dimensions of the toroidal k-ary grid, hence the aim is to move from the pattern of the source node through the appropriate intermediate nodes in order to keep adapting to the pattern of the destination node, where the gap is always one, as in a toroidal k-ary grid there is always a direct link to any node whose Hamming distance is one, thus meaning that there is always a direct path to any node with just one discordant symbol, whichever that symbol may be. Hence, variable gap may be skipped in the algorithms, which simplifies them a lot. On the other hand, if more than one move must be made, then redundant paths of minimal cost may be achieved by combining the different movements to adapt to the destination pattern, where such paths may be visually found by following the needed intermediate patterns.

For example, taking the toroidal k-ary grid for $k = 3$ and $n = 3$, in order to move from node 000 to node 012, each pair of corresponding symbols must be studied. The leftmost symbol is the same in both cases, so no movement must be made in that dimension, as actually both points are over the same layer. The middle and right symbols are different, thus the Hamming distance between both nodes is 2. Hence, a possible path could be to first adapt the middle symbol, thus moving to node 010, and in turn, adapting the rightmost symbol, thus reaching node 012. On the other hand, another possible path could be to initially adapt the rightmost symbol, thus going to node 002, and then, adapting the middle symbol, but getting to node 012. Consequently, both paths are redundant and contain just 2 hops.

Therefore, in order to implement algorithms to model the behavior of toroidal k-ary grids, both the distributed one and the centralized one, it is necessary to create a function to map the patterns established throughout the toroidal k-ary grid, which will depend on each particular instance. Hence, the algorithm will check each pair of corresponding symbols between the source node and the destination node, and each time a discordant symbol is found, then a function

called PatternMapping(a_j,b_j) will be called in order to choose the neighbor node to move in so as to adapt that symbol from the source node, meaning a_j, to the proper symbol in the destination node, meaning b_j. However, that function is not going to be coded as patterns found in each dimension depend on the instance of toroidal k-ary grid. Furthermore, as stated in the k-ary n-cube, a randomization function could also be implemented in the distributed algorithm in order not to always check each dimension in a sequential order, thus always dealing first with the lower dimensions, but in a random fashion, thus choosing a different order of dimensions at each execution, although this is not stated in the algorithms proposed.

Algorithm 3 Distributed algorithm to go from node a to node b within a toroidal k-ary grid.

Require: a, b, k, n
  $flag \leftarrow 0$
  for $j \leftarrow 0$ to $n-1$ **do**
    while $flag == 0$ **do**
      $a_j \leftarrow \left\lfloor \frac{a}{k^j} \right\rfloor_{|k}$
      $b_j \leftarrow \left\lfloor \frac{b}{k^j} \right\rfloor_{|k}$
      if $a_j != b_j$ **then**
        $PatternMapping(a_j, b_j)$
        $flag \leftarrow 1$
      end if
    end while
  end for

Algorithm 4 Centralized algorithm to go from node a to node b within a toroidal k-ary grid.

Require: a, b, k, n
  for $j \leftarrow 0$ to $n-1$ **do**
    $a_j \leftarrow \left\lfloor \frac{a}{k^j} \right\rfloor_{|k}$
    $b_j \leftarrow \left\lfloor \frac{b}{k^j} \right\rfloor_{|k}$
    if $a_j != b_j$ **then**
      $PatternMapping(a_j, b_j)$
    end if
  end for

7. Conclusions

In this Paper, an arithmetic modeling of k-ary n-cubes and toroidal k-ary grids have been exposed. To start with, a toroidal structure has been defined, focusing on the 2D-torus, where three types of representation have been exhibited, such as its embedding in an $\mathbb{R}^4$, which is the most straightforward, followed by its embedding in an $\mathbb{R}^3$ space, which is the most commonly used, and its embedding in an $\mathbb{R}^2$, which is the most convenient for modeling toroidal topologies.

Afterwards, the concept of norm and distance have been reviewed, where the most common norms have been described, along with its induced distances. In this sense, l_0-norm induces the Hamming distance, which accounts for the number of different pairs of components between a couple of vectors. Then, l_1-norm induces the Manhattan distance, which stands for the orthogonal movements to move between two points. Next, l_2-norm induces the Euclidean distance, which represents the shortest distance between two points, it is based on the extension of the Pythagorean Theorem and it is the most popular one. Also, l_∞-norm induces the

Chebyshev distance, which yields the maximum difference in absolute value between the different pairs of components of two points.

After that, toroidal topologies have been classified between structures whose aim is pattern spotting, such as the different instances of de Bruijn shapes, and the structures whose target is to implement a node identification scheme to optimize either resource migration or movement strategies among nodes. Next, de Bruijn shapes have been outlined in order to present their main features and how they are used for locating unique patterns across the layout.

At this point, k-ary n-cubes have been introduced as an easy way to assign node identifiers in a sequential manner. This layout allows to know the exact location of a given source node in each of the n dimensions of the structure by means of integer divisions and arithmetic modulo k, which may also be applied to find out the moves to be made through the proper intermediate nodes so as to reach any given destination node. In this sense, a particular node identifier in decimal notation may be turned into base-k notation, where each symbol represents the value associated with a determined dimension.

The comparisons between the corresponding pairs of symbols identifying two nodes may be done in an arithmetic manner by applying integer divisions and arithmetic modulo k, or alternatively, in a logic fashion by applying the XOR logic operation. Anyway, two algorithms have been proposed so as to model a k-ary n-cube in pseudocode, where one is distributed and the other one is centralized. Regarding the former, it allows for a given node to indicate the next node on the shortest way towards a given destination node, whilst with respect to the latter, it permits for a particular node to denote the full path to reach a given destination node.

An alternative design to identify nodes is the toroidal k-ary grid, which is only available in a few instances, and whose aim is to get all nodes with a Hamming distance of one at just one hop away, which is not always the case in k-ary n-grid designs. This alternative way of node identification is not based on arithmetic operations, as it is the case of k-ary n-cube, but it instead relays on mapping patterns along the different dimensions of each node. This way, the movement from a given source node to a given destination node must start by searching for a neighbor node whose pattern for a given symbol matches the pattern of the destination node for that symbol, and subsequently, that next node will search for another symbol showing in the pattern of the destination node and not in that next node, and this process will keep going on for further nodes until reaching the pattern of the destination node, thus indeed getting there.

One of the issues of this layout is that its few instances do not share a common metapattern among them, but each one has its own particular pattern. Hence a unique function to map patterns found for all dimensions in each node must be implemented in order to be able to identify the moves to go from a source node to a destination node by adapting the pattern in the source to the pattern in the destination, which is done by making the appropriate hops where the necessary intermediate patterns are located.

As that function called PatternMapping() do depend on the toroidal k-ary grid instance, it has not been defined in the algorithms for this topology, although it have been stated as a call to that function, whose target is to find a neighbor node where its pattern for a given symbol is different for that of the source node in order to make it equal to that of the destination node. Anyway, two algorithms have been proposed, as in the k-ary n-cube case, where the distributed one only searches a single next hop on the way to the destination node, thus mirroring the behavior of how routers and switches work, whereas the centralized one finds the whole path to reach the destination node, no matter how many intermediate nodes are involved, thus following the behavior of the way a network controller and an orchestrator work.

Eventually, it is to be noted that the algorithms to model k-ary n-cubes are longer because some lines of code are devoted to process the gap between each corresponding pair of symbols a_j and b_j by means of arithmetic operations, as that is the key point of those algorithms. On the other hand, the algorithms to model toroidal k-ary grids are shorter because such a gap

is always one, even though the computational cost of the function PatternMapping() will grow with the size of the toroidal structure. Hence, the algorithm with the best performance may depend on features k and n, as well as on the way in which the pattern function is implemented.

ACKNOWLEDGEMENTS

This research work is part of the TED2021-132695B-I00 project, financed by MCIN/AEI/10.13039/501100011033 and by the European Union "NextGenerationEU"/PRTR.

References

[1] Wang T, Dai Z, Potier-Ferry M and Xu F 2023 Curvature-Regulated Multiphase Patterns in Tori *Physical Review Letters* **130** 048201.
[2] Kallinikos N, McKay R S and Syndercombe T 2022 Regions without invariant tori of given class for the planar circular restricted three-body problem *Physica D: Nonlinear Phenomena* **434** 133216.
[3] Wang T, Su Z, Xia Y, Qin B and Hamdi M 2014 NovaCube: A low latency Torus-based network architecture for data centers *Proceedings of the IEEE Global Communications Conference* 2252–2257.
[4] Kim D S, Kim Y H and Qian J 2020 Characterization of Clifford Torus in Three-Spheres *Mathematics* **8(5)** 0718.
[5] Meijer D, Wong K W and Geesink H 2022 Primordial Configuration Space: Discrete Frequency Patterns of Phonons Reveal a Phase Space with Chern-Invariant Metrics and Acoustic Signature (published in ResearchGate)
[6] Bossard A 2022 Topological properties of a 2-dimensional torus-connected cycles network *Proceedings of ICCM'22* 139–144.
[7] Sharma A and Gopalan S R 2021 Performance analysis of torus optical interconnect with data center traffic *ETRI Journal* **43(1)** 64–73.
[8] Gardner R J, Hermansen E, Pachitariu M, Burak Y, Baas N A, Dunn B A, Moser M B and Moser E I 2022 Toroidal topology of population activity in grid cells *Nature* **602** 123–128.
[9] Borrelli V, Jabrane S, Lazarus F and Thibert B 2012 Flat tori in three-dimensional space and convex integration *PNAS* **109(19)** 7218–7223.
[10] Roig P J 2022 *Formal Algebraic Modelling for Fog Computing Network Architecture* (PhD Thesis in Information and Communication Technology: University of the Balearic Islands)
[11] Murdzek R 2007 The Geometry of the Torus Universe *International Journal of Modern Physics* **16(04)** 681–686.
[12] l0-Norm, l1-Norm, l2-Norm, l-infinity Norm (accessed on 15th August 2023). URL: https://rorasa.wordpress.com/2012/05/13/l0-norm-l1-norm-l2-norm-l-infinity-norm/
[13] Roig P J, Alcaraz S, Gilly K, Bernad C and Juiz C 2021 Review on de Bruijn shapes in one, two and three dimensions *Journal of Physics: Conference Series* **2090** 012047
[14] de Bruijn NG 1946 A combinatorial problem *Proceedings of the Section of Sciences of the Koninklijke Nederlandse Akademie van Wetenschappen te Amsterdam* **49(7)** 758–764.
[15] Roig P J, Alcaraz S, Gilly K, Bernad C and Juiz C 2022 De Bruijn-based and k-ary n-cube-based algebraic models in fog environments *Communications in Computer and Information Science* **1521** 126–141.
[16] Xie Y, Liang J, Yin W and Li C 2022 The properties and t/s-diagnosability of k-ary n-cube networks *Journal of Supercomputing* **78(5)** 7038–7057.
[17] Roig P J, Alcaraz S, Gilly K, Bernad C and Juiz C 2023 Approach to Layouts Used in Toroidal Topologies *Advances and Challenges in Science and Technology* **9** 81–97.